



Lab Assessment Task -I - Agentic AI System using Modern LLM-based Tools

“Academic Planner”

by

16010123012: Aaryan Sharma

16010123017: Aditey Kshirsagar

16010123023: Aditya Baheti

For the subject

Artificial Intelligence

Department of Computer Engineering

Academic Year 2025-26

Topic Name - Academic Planner Agent

Introduction of problem statement

Agentic AI systems represent a paradigm shift from traditional rule-based systems to intelligent systems capable of reasoning, planning, and tool usage. Unlike classical agents, modern agents leverage Large Language Models (LLMs) to dynamically interpret goals and execute multi-step actions.

This project implements an Academic Planner Agent that uses an LLM along with algorithmic tools to generate optimized study schedules based on constraints such as deadlines, subject difficulty, and available time.

Abstract of problem statement

This project presents an LLM-based Academic Planner Agent designed using modern agentic AI principles. The system accepts structured student input and generates an optimized study plan using a hybrid approach combining deterministic algorithms and LLM reasoning.

The agent employs:

- Priority-based scheduling algorithms
- ReAct-based reasoning
- Tool integration (calculator, JSON storage, LLM API)

The implementation uses OpenRouter API with LLaMA 3.1 8B, enabling cost-effective and efficient reasoning. The system demonstrates multi-step planning, contextual memory, and intelligent recommendations, showcasing the evolution from classical AI agents to modern agentic systems.

AI framework(s) used:

- ReAct Agent Pattern
- OpenRouter API (LLM Gateway)
- Custom Python Agent Framework (lightweight)

Tools integrated with agent:

Tool	Description	Why Used
Priority Calculator	Computes urgency scores	Enables intelligent scheduling
Schedule Generator	Creates structured study plan	Core planning logic
OpenRouter LLM	Provides reasoning + suggestions	Adds intelligence layer
JSON Storage	Saves plans	Enables persistence

Part A – Agent Design (Modern View)

1. Goal of the Agent

Primary Goal: Generate optimized study schedules that maximize learning outcomes given limited time and multiple subjects with varying deadlines.

Objectives:

1. Calculate priority scores for subjects based on urgency and difficulty
2. Allocate available study hours efficiently across multiple days
3. Provide intelligent recommendations using LLM reasoning
4. Save plans for future reference

2. Agent Role

The Academic Planner Agent acts as an intelligent study advisor that:

- Understands student constraints (available hours, deadlines)
- Reasons about optimal time allocation
- Provides personalized recommendations
- Adapts to different academic scenarios

3. Tools Used

Our agent integrates 4 specialized tools:

Tool 1: Priority Calculator

Purpose: Calculate urgency scores for each subject

Algorithm:

$\text{priority_score} = \text{difficulty_level} / \text{days_until_deadline}$

Rationale: Subjects that are both difficult and have approaching deadlines receive higher priority.

Example:

- AI: difficulty=4, deadline=2 days → priority = $4/2 = 2.0$
- Math: difficulty=5, deadline=3 days → priority = $5/3 = 1.67$
- OS: difficulty=3, deadline=5 days → priority = $3/5 = 0.6$

Tool 2: Schedule Generator

Purpose: Create day-by-day study plan

Input: Sorted priorities, available hours per day, planning window

Output: Structured schedule allocating hours to each subject per day

Algorithm:

- For each day, allocate hours to subjects in priority order
- Limit allocation per subject to prevent monotony
- Respect daily time constraints

Tool 3: LLM Agent Call (OpenRouter API)

Purpose: Generate intelligent recommendations using large language model

Model: meta-llama/llama-3.1-8b-instruct

Integration: REST API via HTTP POST requests

What it provides:

- Contextual study advice
- Analysis of the generated plan
- Suggestions for improvement
- Study technique recommendations

Tool 4: JSON Storage

Purpose: Persist generated plans for future reference

Format: JSON (human-readable, machine-parseable)

Contents: Complete plan with metadata, priorities, schedule, recommendations

4. Memory Type

Our agent implements two-level memory:

Short-term (Contextual) Memory

- Stored in: Python variables during execution
- Duration: Current session only
- Contents:
 - User input (subjects, deadlines, hours)
 - Calculated priorities
 - Generated schedule
 - LLM recommendations

Long-term (Persistent) Memory

- Stored in: `study\_plan.json` file
- Duration: Permanent (until manually deleted)
- Contents:
 - Complete plan with timestamp
 - All inputs and outputs
 - Metadata for tracking

Comparison to Classical Agents:

Classical agents typically have no memory; each query is processed independently. Our modern agent maintains context throughout execution and can save results for future reference.

5. Planning Strategy: ReAct Pattern

We implement the **ReAct (Reasoning + Acting)** pattern, which interleaves:

THOUGHT (Reasoning) → ACTION (Tool Call) → OBSERVATION (Result) → Repeat

Our Agent's ReAct Cycle:

Step 1:

- THOUGHT: "I need to create an optimized study plan"
- ACTION: Analyze input data

Step 2:

- THOUGHT: "First, calculate which subjects are most urgent"
- ACTION: Use Priority Calculator Tool
- OBSERVATION: Priority scores computed

Step 3:

- THOUGHT: "Now generate a day-by-day schedule"
- ACTION: Use Schedule Generator Tool
- OBSERVATION: 5-day schedule created

Step 4:

- THOUGHT: "Let me get AI recommendations"
- ACTION: Call LLM Agent via OpenRouter API
- OBSERVATION: Recommendations received

Step 5:

- THOUGHT: "Save this plan for future reference"
- ACTION: Use JSON Storage Tool
- OBSERVATION: Plan saved successfully

Step 6:

- THOUGHT: "All steps complete, provide final answer"

Benefits of ReAct:

- Makes reasoning visible and traceable
- Enables debugging and validation
- Supports complex multi-step tasks
- Allows iterative refinement

ReAct is chosen because it enables interleaving reasoning and action, allowing the agent to dynamically decide when to use tools instead of relying on fixed pipelines.

Part B – Implementation (Modern AI Tools)

- ***LLM-based agent : Explain the LLM used and justify why it was chosen***

We used: OpenRouter + LLaMA 3.1 8B

Why this model?

- Free and accessible
- Strong reasoning capability
- Good for structured outputs
- Lightweight compared to larger models

Tools used: List of tools, justification of choice, features, etc**1. Priority Calculator**

- Converts subjective inputs into quantitative priority scores
- Ensures objective scheduling

2. Schedule Generator

- Implements constraint-based allocation
- Prevents overloading a single subject

3. LLM Tool

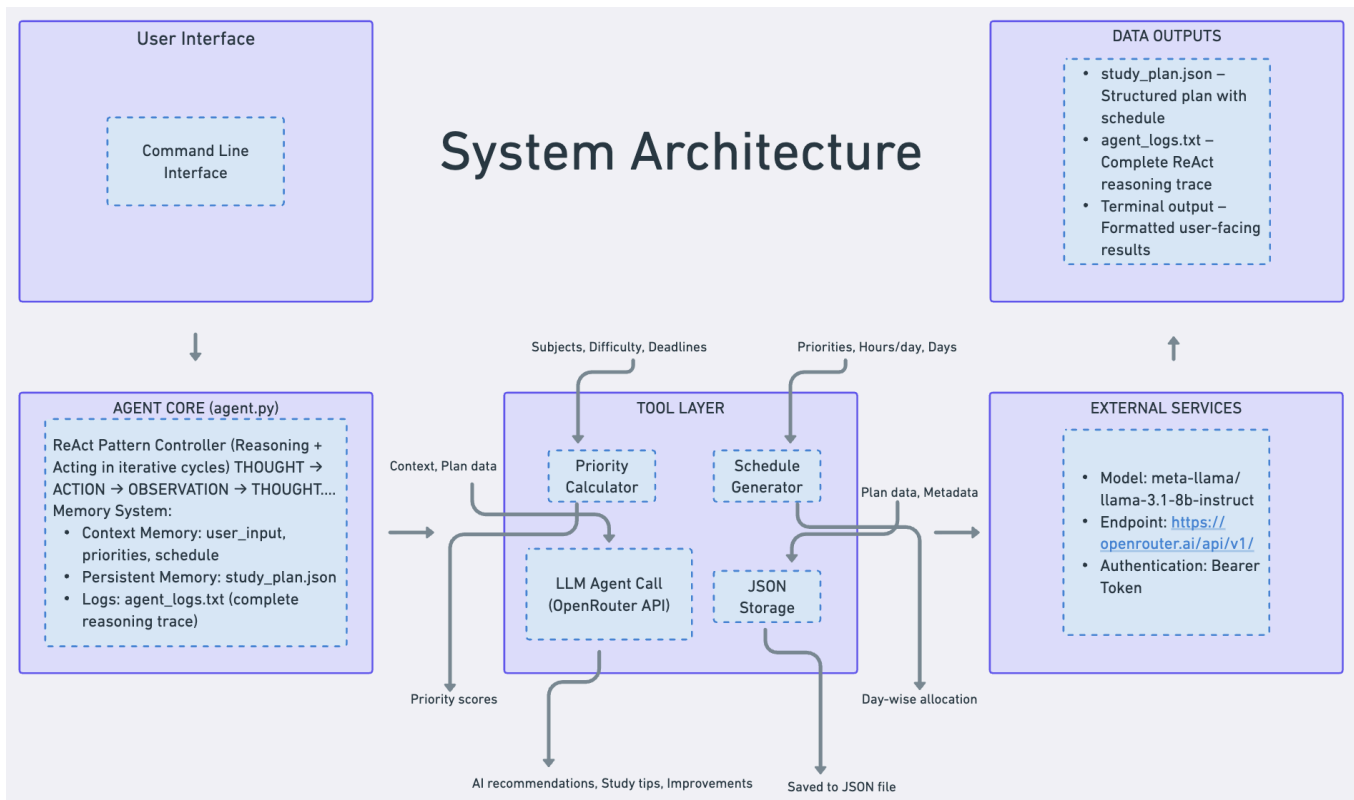
- Adds human-like reasoning
- Improves plan quality with insights

4. JSON Storage

- Enables long-term memory
- Useful for comparison and tracking

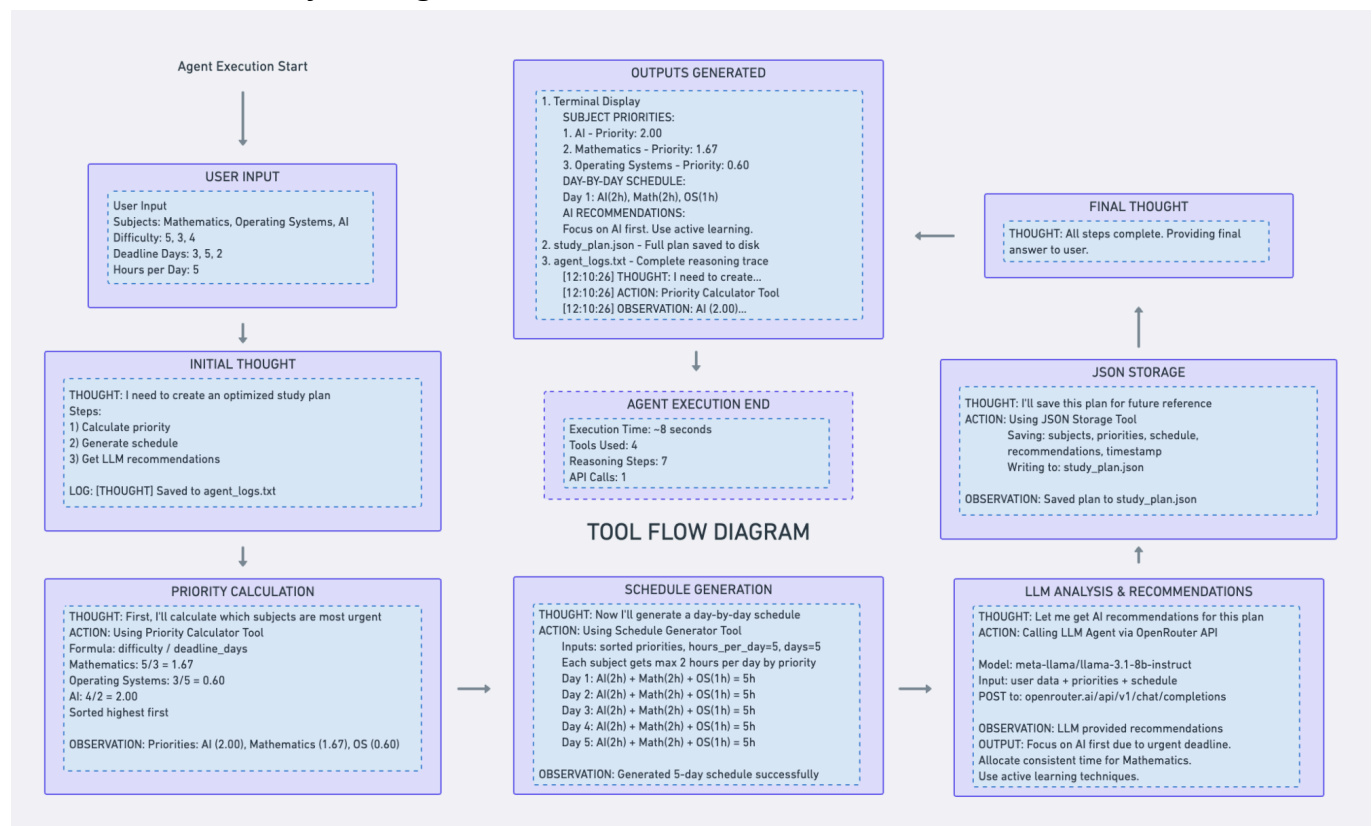
Multi-step reasoning:

- ***Architecture diagram***



Architecture follows a modular pipeline where the LLM orchestrates tool usage while maintaining context memory, enabling dynamic multi-step planning.

o Tool flow diagram



Observable agent behavior (logs / traces):

Code repository / notebook: <https://github.com/Aaryan-Sharma-5/academic-planner>

Agent interaction logs

[15:53:51] THOUGHT

I need to create an optimized study plan. Steps: 1) Calculate priority, 2) Generate schedule, 3) Get LLM recommendations

[15:53:51] INPUT

Subjects: 4, Hours/day: 6

[15:53:51] THOUGHT

First, I'll calculate which subjects are most urgent

[15:53:51] ACTION

Using Priority Calculator Tool

[15:53:51] OBSERVATION

Priorities: DSIP (1.00), NIS (0.60), CCAV (0.57), AI (0.50)

[15:53:51] THOUGHT

Now I'll generate a day-by-day schedule

[15:53:51] ACTION

Using Schedule Generator Tool

[15:53:51] OBSERVATION

Generated 5-day schedule

[15:53:51] THOUGHT

Let me get AI recommendations for this plan

[15:53:51] ACTION

Calling LLM Agent for analysis

[15:54:09] OBSERVATION

LLM provided recommendations

[15:54:09] THOUGHT

I'll save this plan for future reference

[15:54:09] ACTION

Using JSON Storage Tool

[15:54:09] OBSERVATION

Saved plan to logs/study_plan.json

[15:54:09] THOUGHT

All steps complete. Providing final answer to user.

prompts and output screenshots

```
(venv) PS D:\Coding\Project\academic-planner> python agent.py

ACADEMIC PLANNER AGENT
Lab CA-Activity 1: Agentic AI System

=====
ACADEMIC PLANNER AGENT - Input Collection
=====

How many subjects do you need to plan for? 4

--- Subject 1 ---
  Subject name: DSIP
  Difficulty level (1-5): 4
  Days until deadline: 4

--- Subject 2 ---
  Subject name: NIS
  Difficulty level (1-5): 3
  Days until deadline: 5

--- Subject 3 ---
  Subject name: AI
  Difficulty level (1-5): 3
  Days until deadline: 6

--- Subject 4 ---
  Subject name: CCAV
  Difficulty level (1-5): 4
  Days until deadline: 7

How many hours can you study per day? 6
```

```
=====
Your Input:
=====
```

```
{
  "subjects": [
    {
      "name": "DSIP",
      "difficulty": 4,
      "deadline_days": 4
    },
    {
      "name": "NIS",
      "difficulty": 3,
      "deadline_days": 5
    },
    {
      "name": "AI",
      "difficulty": 3,
      "deadline_days": 6
    },
    {
      "name": "CCAV",
      "difficulty": 4,
      "deadline_days": 7
    }
  ],
  "hours_per_day": 6
}
```

```
=====
ACADEMIC PLANNER AGENT - ReAct Pattern
=====
```

```
[15:53:51] THOUGHT
  I need to create an optimized study plan. Steps: 1) Calculate priority, 2) Generate schedule, 3) Get LLM recommendations
[15:53:51] INPUT
  Subjects: 4, Hours/day: 6
[15:53:51] THOUGHT
  First, I'll calculate which subjects are most urgent
[15:53:51] ACTION
  Using Priority Calculator Tool
[15:53:51] OBSERVATION
  Priorities: DSIP (1.00), NIS (0.60), CCAV (0.57), AI (0.50)
[15:53:51] THOUGHT
  Now I'll generate a day-by-day schedule
[15:53:51] ACTION
  Using Schedule Generator Tool
[15:53:51] OBSERVATION
  Generated 5-day schedule
[15:53:51] THOUGHT
  Let me get AI recommendations for this plan
[15:53:51] ACTION
  Calling LLM Agent for analysis
[15:54:09] OBSERVATION
  LLM provided recommendations
[15:54:09] THOUGHT
  I'll save this plan for future reference
[15:54:09] ACTION
  Using JSON Storage Tool
[15:54:09] OBSERVATION
  Saved plan to logs/study_plan.json
[15:54:09] THOUGHT
  All steps complete. Providing final answer to user.
```

FINAL STUDY PLAN

SUBJECT PRIORITIES

1. DSIP - Priority: 1.00 [*****]
Difficulty: 4/5 | Deadline: 4 days
2. NIS - Priority: 0.60 [***]
Difficulty: 3/5 | Deadline: 5 days
3. CCAV - Priority: 0.57 [**]
Difficulty: 4/5 | Deadline: 7 days
4. AI - Priority: 0.50 [**]
Difficulty: 3/5 | Deadline: 6 days

DAY-BY-DAY SCHEDULE

Day 1: (6 hours total)

- > DSIP 2h - Focus on DSIP fundamentals
- > NIS 2h - Focus on NIS fundamentals
- > CCAV 2h - Focus on CCAV fundamentals

Day 2: (6 hours total)

- > DSIP 2h - Focus on DSIP fundamentals
- > NIS 2h - Focus on NIS fundamentals
- > CCAV 2h - Focus on CCAV fundamentals

Day 3: (6 hours total)

- > DSIP 2h - Focus on DSIP fundamentals
- > NIS 2h - Focus on NIS fundamentals
- > CCAV 2h - Focus on CCAV fundamentals

Day 4: (6 hours total)

- > DSIP 2h - Focus on DSIP fundamentals
- > NIS 2h - Focus on NIS fundamentals
- > CCAV 2h - Focus on CCAV fundamentals

Day 5: (6 hours total)

- > DSIP 2h - Focus on DSIP fundamentals
- > NIS 2h - Focus on NIS fundamentals
- > CCAV 2h - Focus on CCAV fundamentals

AI RECOMMENDATIONS

****Why this prioritization makes sense:****

The generated schedule prioritizes subjects with higher deadlines and difficulty levels, ensuring that the most challenging and time-sensitive topics are addressed first. The distribution of hours across multiple subjects also seems reasonable, allowing for a balanced study routine. However, the lack of variance in the daily schedule may not accommodate different learning styles or personal productivity rhythms.

****Suggestions for improvement:****

1. ****Adjust daily schedule:**** Consider breaking up study sessions into focused blocks (e.g., 90 minutes of intense study + 10-15 minute breaks) to optimize learning retention and productivity.

2. ****Incorporate active learning:**** Replace some of the "Focus on [subject] fundamentals" topics with more engaging and interactive learning materials, such as practice problems, quizzes, or peer discussions.

3. ****Review and adjust priorities:**** Regularly reassess the priority levels and deadlines of each subject to ensure the schedule remains aligned with the learner's needs and goals.

****Study tips:****

* ****Pomodoro technique:**** Alternate between focused study sessions (e.g., 90 minutes) and scheduled breaks to maintain productivity and reduce burnout.

* ****Active recall:**** Regularly attempt to recall key concepts and information without relying on notes or other external resources to reinforce long-term understanding.

* ****Review and reflection:**** Set aside dedicated time for reviewing progress, identifying areas for improvement, and adjusting the study plan as needed.

Screen recording link:

<https://drive.google.com/file/d/15DS7FAwxQEH3YHqv4WQbGEkhrPTHbW9w/view?usp=sharing>

Discussion, takeaway

- LLMs significantly enhance traditional agents by enabling reasoning
- Combining deterministic logic + LLM = best performance
- ReAct framework improves transparency and debugging
- Tool integration is essential for real-world AI systems

Key Insight:

Pure LLM is not enough. Hybrid systems are more reliable.

Limitations:

- The system does not dynamically re-plan if a study session is missed
- Accuracy depends on correctness of user-provided inputs

Conclusion

The Academic Planner Agent successfully demonstrates the capabilities of modern agentic AI systems. By integrating algorithmic tools with LLM-based reasoning, the system generates efficient and adaptive study schedules.

The project highlights:

- Transition from static to dynamic AI systems
- Importance of multi-step reasoning
- Role of tool orchestration

This implementation validates that LLM-powered agents can solve real-world planning problems effectively and efficiently.

References

1. Yao, S., et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629
2. Wang, L., et al. (2023). *A Survey on Large Language Model based Autonomous Agents*. arXiv:2308.11432
3. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.)
4. OpenRouter Documentation. (2026). <https://openrouter.ai/docs>
5. Meta AI. (2024). *Llama 3.1: Open Source LLM*